# Assignment -03

## Experiment 3.1: Conditional Statements

**Q1. WAP to take check if the triangle is valid or not. If the validity is established, do check if the triangle is isosceles, equilateral, right angle, or scalene. Take sides of the triangle as input from a user.**

Code:

```c
//WAP to take check if the triangle is valid or not. If the validity is established, do check if the triangle is isosceles,
//equilateral, right angle, or scalene. Take sides of the triangle as input from a user.
#include <stdio.h>
int main(){
    int a ,b ,c;

    //input the sides of the triangle
    printf("Enter the sides of the triangle: ");
    scanf("%d %d %d", &a, &b, &c);

    //calculates and prints the result
    if(a + b > c && a + c > b && b + c > a){
        printf("The triangle is valid.\n", a,b,c);

        if(a==b && b==c && a==c){
            printf("The triangle is equilateral.", a,b,c);
        }

        else if(a==b || b==c || a==c){
            printf("The triangle is isosceles.", a,b,c);
        }

        else if(a*a + b*b == c*c || b*b + c*c == a*a || a*a + c*c == b*b){
            printf("The triangle is right angled.", a,b,c);
        }

        else{
            printf("The triangle is scalene.");
        }

    }
    else{
        printf("The triangle is not valid.");
    }

    return 0;
}
```

Output:

```
Enter the sides of the triangle: 2 3 4
The triangle is valid.
The triangle is scalene.
```

Summary:

- A triangle is valid if sum of any two sides > third side.
- Types:

o Equilateral → all sides equal.
o Isosceles → any two sides equal.
o Right-angled → Pythagoras theorem holds.
o Scalene → all sides different.

- Program assumes integer input only. If user enters decimals (e.g., 3.5), result may be incorrect.
- Does not check for negative or zero values.

Q2. WAP to compute the BMI Index of the person and print the BMI values as per the given ranges.

Code:

```c
#include <stdio.h>
int main()
{
    float weight, height, BMI;

    printf("Enter the weight (in kgs): ");
    scanf("%f", &weight);

    printf("Enter the height (in mts): ");
    scanf("%f", &height);

    BMI = weight / (height * height);

    if(BMI<15)
    {
        printf("Starvation");
    }

    else if(BMI>=15.1 && BMI<=17.5)
    {
        printf("Anorexic");
    }

    else if(BMI>=17.6 && BMI<=18.5)
    {
        printf("Underweight");
    }

    else if(BMI>=18.6 && BMI<=24.9)
    {
        printf("Ideal");
    }

    else if(BMI>=25 && BMI<=25.9)
    {
        printf("Overweight");
    }
```

```c
    else if(BMI>=30 && BMI<=39.9)
    {
        printf("Obese");
    }

    else if(BMI>=40.0 )
    {
        printf("Morbidity Obese");
    }

    return 0;
}
```

Output:

```
Enter the weight (in kgs): 45
Enter the height (in mts): 1.50
Ideal
```

Summary:

- Formula: BMI = weight (kg) / (height × height).
- BMI is a health index indicating if a person is underweight, healthy, or overweight.
- Categories are based on WHO guidelines.
- BMI does not consider muscle vs. fat composition → may misclassify athletes.
- Height input of 0 causes division by zero error.
- Only works for metric system (kg, m).

## Q3. WAP to check if three points (x1, y1), (x2, y2), and (x3, y3) are collinear or not.

Code:

```c
#include <stdio.h>
int main()
{
    float x1, y1, x2, y2, x3, y3;

    //input the coordinates of point A,B,C
    printf("Enter coordinates of point A (x1,y1): ");
    scanf("%f,%f", &x1, &y1);

    printf("Enter coordinates of point B (x2,y2): ");
    scanf("%f,%f", &x2, &y2);

    printf("Enter coordinates of point C (x3,y3): ");
    scanf("%f,%f", &x3, &y3);

    //calculates the result
    float area = 0.5 * ((x1*(y2 - y3)) + (x2*(y3 - y1)) + (x3*(y1 - y2)));

    //prints the output
    if (area == 0)
        printf("The points are collinear.\n");
    else
        printf("The points are not collinear.\n");

    return 0;

}
```

Output:

```
Enter coordinates of point A (x1,y1): 2,3
Enter coordinates of point B (x2,y2): 3,4
Enter coordinates of point C (x3,y3): 4,6
The points are not collinear.
```

Summary:

- If area of triangle formed by 3 points = 0, then the points are collinear.
- Formula: Area = x1(y2- y3) + x2(y3- y1) + x3(y1- y2).
- Works with integers; may give precision issues with floating-point coordinates.
- Does not handle extremely large values where multiplication may overflow.

## Q4. According to the Gregorian calendar, it was Monday on 01/01/01. If any year is input through the keyboard, write a program to find out what is the day on 1st January of this year.

Code:

```c
#include <stdio.h>

int main() {
    int year;

    //input the year
    printf("Enter year: ");
    scanf("%d", &year);

    // Days of the week starting from Monday
    char *days[] = {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"};

    // Count total days from year 1 to (year-1)
    long total_days = 0;
    for (int i = 1; i < year; i++) {
        // Leap year check
        if ((i % 400 == 0) || (i % 4 == 0 && i % 100 != 0))
            total_days += 366;
        else
            total_days += 365;
    }

    // Day index: starting from Monday (index 0)
    int day_index = total_days % 7;

    //prints the output
    printf("1st January %d is %s\n", year, days[day_index]);

    return 0;
}
```

Output:

```
-o c4 } ; if ($?) { .\c4 }
Enter year: 2025
1st January 2025 is Wednesday
```

Summary:

- `% 7` ensures result is within 0–6 (days of week).
- Valid only for Gregorian calendar (after 1582)
- Input before year 1 (like 0 or negatives) is invalid.
- Program assumes year ≥ 1 and does not validate user input.

Q5. WAP using ternary operator, the user should input the length and breadth of a rectangle, one has to find out which rectangle has the highest perimeter. The minimum number of rectangles should be three.

Code:

```c
#include <stdio.h>

int main() {
    int l1, b1, l2, b2, l3, b3;
    int p1, p2, p3;

    //input the length and breadth of all three rectangles
    printf("Enter length and breadth of Rectangle 1: ");
    scanf("%d %d", &l1, &b1);

    printf("Enter length and breadth of Rectangle 2: ");
    scanf("%d %d", &l2, &b2);

    printf("Enter length and breadth of Rectangle 3: ");
    scanf("%d %d", &l3, &b3);

    //calculates the result
    p1 = 2 * (l1 + b1);
    p2 = 2 * (l2 + b2);
    p3 = 2 * (l3 + b3);

    int max = (p1 > p2) ? ((p1 > p3) ? p1 : p3) : ((p2 > p3) ? p2 : p3);

    //prints the result
    printf("The maximum perimeter is: %d\n", max);

    return 0;
}
```

Output:

```
 -o c5 } ; if ($?) { .\c5 }
Enter length and breadth of Rectangle 1: 5 10
Enter length and breadth of Rectangle 2: 7 12
Enter length and breadth of Rectangle 3: 6 8
The maximum perimeter is: 38
```

Summary:

- Perimeter formula: 2 × (length + breadth).
- The **ternary operator (? :)** is used to select the maximum value among three perimeters.
- Useful for concise conditional logic.
- Only works for exactly **3 rectangles**.
- Negative or zero inputs not handled.
- If two perimeters are equal and maximum, program doesn't tell which rectangle(s) have it.

# Experiment 3.2: Loops

## Q1. WAP to enter numbers till the user wants. At the end, it should display the count of positive, negative, and Zeroes entered.

Code:

```c
#include <stdio.h>

int main() {
    int num;
    int pos = 0, neg = 0, zero = 0;
    char choice;

    do {
        //input the number
        printf("Enter a number: ");
        scanf("%d", &num);

        if (num > 0)
            pos++;
        else if (num < 0)
            neg++;
        else
            zero++;

        printf("Do you want to enter another number? (y/n): ");
        scanf(" %c", &choice);

    } while (choice == 'y' || choice == 'Y');

    //prints the result
    printf("Result");
    printf("Positive numbers: %d\n", pos);
    printf("Negative numbers: %d\n", neg);
    printf("Zeroes: %d\n", zero);

    return 0;
}
```

Output:

```
Enter a number: 2
Do you want to enter another number? (y/n): y
Enter a number: -1
Do you want to enter another number? (y/n): y
Enter a number: 0
Do you want to enter another number? (y/n): n
ResultPositive numbers: 1
Negative numbers: 1
Zeroes: 1
```

Summary:

- The program repeatedly asks the user to enter numbers.
- After each number, it asks whether the user wants to continue.
- It counts how many numbers are positive, negative, and zero.
- When the user chooses to stop, it displays the final counts.

## Q2. WAP to print the multiplication table of the number entered by the user. It should be in the correct formatting. Num * 1 = Num.

Code:

```c
#include <stdio.h>

int main() {
    int num;

    //input the number
    printf("Enter a number: ");
    scanf("%d", &num);

    printf("Multiplication Table of %d:\n", num);

    //calculates and prints the result
    for (int i = 1; i <= 10; i++) {
        printf("%d * %d = %d\n", num, i, num * i);
    }

    return 0;
}
```

Output:

```
gcc c2.c -o c2 } ; if ($?) { .\c2 }
Enter a number: 3
Multiplication Table of 3:
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
```

Summary:

- Each step shows the calculation in the format *num * i = result*.

## Q3. WAP to generate the following set of output.

a. 1

2 3

4 5 6

Code:

```c
#include <stdio.h>

int main() {
    int num = 1;
    int rows;

    //input the number of rows
    printf("Enter the number of rows: ");
    scanf("%d", &rows);

    //calculates and prints the result
    for (int i = 1; i <= rows; i++) {    // rows
        for (int spaces = rows - i; spaces > 0; spaces--){
            printf(" ");
        }

        for (int j = 1; j <= i; j++) {   // numbers
            printf("%d ", num);
            num++;
        }

        printf("\n");
    }

    return 0;
}
```

Output:

```
Enter the number of rows: 3
  1
 2 3
4 5 6
```

Summary:

- The program prints a number pattern in a triangular format.
- Numbers start from 1 and increase sequentially across rows.
- Each row is properly spaced to align the numbers in a pyramid-like shape.

b. 1

1 1

1 2 1

1 3 3 1

1 4 6 4 1

Code:

```c
#include <stdio.h>

int main() {
    int rows = 5;

    for (int i = 0; i < rows; i++) {
        int num = 1;

        // Print spaces
        for (int spaces = 0; spaces < rows - i - 1; spaces++) {
            printf(" ");
        }

        // Print numbers in the row
        for (int j = 0; j <= i; j++) {
            printf("%d ", num);
            num = num * (i - j) / (j + 1);  // Calculate next number in the row
        }

        printf("\n");
    }

    return 0;
}
```

Output:

```
gcc c3_b.c -o c3_b } ; if ($?) { .\c3_b }
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

Summary:

- The program prints Pascal's Triangle up to 5 rows.
- Each row is properly aligned with spaces to form a pyramid.
- Numbers in each row start with 1, and subsequent numbers are calculated using a simple formula based on the previous number.
- The triangle grows row by row, showing the pattern of combinations.

Q4. The population of a town is 100000. The population has increased steadily at the rate of 10% per year for the last 10 years. Write a program to determine the population at the end of each year in the last decade.

Code:

```c
#include <stdio.h>

int main() {
    double population = 100000;
    double growthRate = 0.10;
    int years = 10;

    printf("Year\tPopulation\n");

    for (int i = 1; i <= years; i++) {
        population = population + population * growthRate; // increase by 10%
        printf("%d\t%.0lf\n", i, population); // display as whole number
    }

    return 0;
}
```

Output:

```
gcc c4.c -o c4 } ; if ($?) { .\c4 }
Year    Population
1       110000
2       121000
3       133100
4       146410
5       161051
6       177156
7       194872
8       214359
9       235795
10      259374
```

Summary:

- The program starts with an initial population of 100,000.
- It increases the population by 10% each year for 10 years.
- At the end of each year, it displays the updated population.
- This shows the population growth over the last decade.

Q5. Ramanujan Number is the smallest number that can be expressed as the sum of two cubes in two different ways. WAP to print all such numbers up to a reasonable limit.
Example of Ramanujan number: 1729 12^3 + 1^3 and 10^3 + 9^3.
for a number L=20(that is limit)

Code:

```c
#include <stdio.h>

int main() {
    int L;

    //input the limit
    printf("Enter the limit: ");
    scanf("%d", &L);

    printf("Ramanujan numbers up to %d:\n", L);

    //calculates and prints the result
    for (int n = 1; n <= L; n++) {
        int count = 0;

        for (int a = 1; a*a*a < n; a++) {
            for (int b = a; a*a*a + b*b*b <= n; b++) {
                if (a*a*a + b*b*b == n) {
                    count++;
                }
            }
        }

        if (count >= 2) {
            printf("%d is a Ramanujan number\n", n);
        }
    }

    return 0;
}
```

Output:

```
gcc c5.c -o c5 } ; if ($?) { .\c5 }
Enter the limit: 2000
Ramanujan numbers up to 2000:
1729 is a Ramanujan number
```

Summary:

- The program checks all numbers up to a given limit L.
- For each number, it finds all pairs of integers (a, b) whose cubes sum to that number.
- If a number can be expressed as the sum of two cubes in **two different ways**, it is identified as a Ramanujan number.
- The program prints all such numbers up to the limit.